

SESSION 13 : Window Functions - Part 1

TASKS

Task 1: Create a table named Playlists with columns: id, user_id, playlist_name, and total_likes. Insert at least 8 sample rows with different users and playlists, making sure some playlists have the same user_id.

Query / Answer:

```
CREATE TABLE Playlists (  
    id INT PRIMARY KEY,  
    user_id INT,  
    playlist_name VARCHAR(100),  
    total_likes INT  
);  
  
)
```

Image / Screenshot:

The screenshot shows a database IDE interface. On the left, a sidebar lists various database objects, including 'Schemas'. The main editor area displays SQL code for creating a table named 'Playlists' and inserting 8 sample rows. The code is as follows:

```
1 id INT PRIMARY KEY,  
2 user_id INT,  
3 playlist_name VARCHAR(100),  
4 total_likes INT  
5 );  
6  
7 INSERT INTO Playlists101 (id, user_id, playlist_name, total_likes)  
8 VALUES  
9 (1, 101, 'Workout Beats', 250),  
10 (2, 102, 'Chill Vibes', 180),  
11 (3, 101, 'Morning Motivation', 320),  
12 (4, 103, 'Party Mix', 450),  
13 (5, 104, 'Romantic Hits', 210),  
14 (6, 102, 'Travel Tunes', 175),  
15 (7, 105, 'LoFi Study', 390),  
16 (8, 101, 'Old Classics', 150);  
17  
18
```

Below the code editor, there is an 'Output' section with a table showing the results of the SQL execution:

#	Time	Action	Message
11	13:48:49	select name from restaurants111 where name like 'Sus%': LIMIT 0, 1000	1 row(s) returned
12	13:51:06	select name from restaurants111 where rating between 3.5 and 4.5 LIMIT 0, 1000	4 row(s) returned

Task 2: Write a SQL query using ROW_NUMBER() and the OVER() clause to assign a unique row number to each playlist, ordered by total_likes in descending order.

Query / Answer:

SELECT

id,

user_id,

playlist_name,

total_likes,

row_number() over (partition by user_id order by total_likes desc) from playlists101;

Image / Screenshot:

g_app

edures

es

ations

ers

ts2

mas

23 playlist_name,

24 total_likes,

25 row_number() over (partition by user_id order by total_likes desc) from playlists101;

Result Grid

Filter Rows:

Exports

Wrap Cell Contents

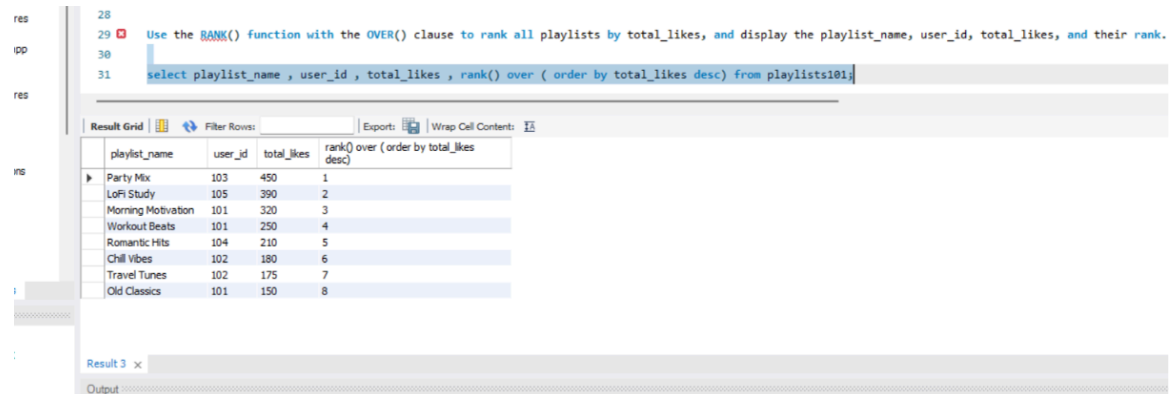
	id	user_id	playlist_name	total_likes	row_number() over (partition by user_id order by total_likes desc)
▶	3	101	Morning Motivation	320	1
	1	101	Workout Beats	250	2
	8	101	Old Classics	150	3
	2	102	Chill Vibes	180	1
	6	102	Travel Tunes	175	2
	4	103	Party Mix	450	1
	5	104	Romantic Hits	210	1
	7	105	LoFi Study	390	1

Task 3: Use the RANK() function with the OVER() clause to rank all playlists by total_likes, and display the playlist_name, user_id, total_likes, and their rank.

Query / Answer:

```
select playlist_name , user_id , total_likes , rank() over ( order by total_likes desc) from  
playlists101;
```

Image / Screenshot:



The screenshot shows a SQL query editor with the following query:

```
select playlist_name , user_id , total_likes , rank() over ( order by total_likes desc) from playlists101;
```

The result grid displays the following data:

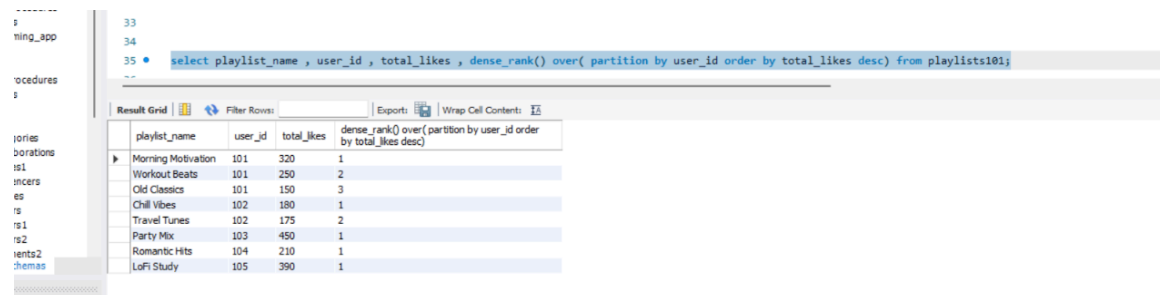
playlist_name	user_id	total_likes	rank() over (order by total_likes desc)
Party Mix	103	450	1
LoFi Study	105	390	2
Morning Motivation	101	320	3
Workout Beats	101	250	4
Romantic Hits	104	210	5
Chill Vibes	102	180	6
Travel Tunes	102	175	7
Old Classics	101	150	8

Task 4: Write a SQL query using DENSE_RANK() and PARTITION BY user_id to rank each user's playlists by total_likes, showing playlist_name, user_id, total_likes, and dense rank.

Query / Answer:

```
select playlist_name , user_id , total_likes , dense_rank() over( partition by user_id order by total_likes desc) from playlists101;
```

Image / Screenshot:



The screenshot shows a SQL query editor with the following query:

```
select playlist_name , user_id , total_likes , dense_rank() over( partition by user_id order by total_likes desc) from playlists101;
```

The results are displayed in a table with the following columns: playlist_name, user_id, total_likes, and dense_rank() over(partition by user_id order by total_likes desc). The results are as follows:

playlist_name	user_id	total_likes	dense_rank() over(partition by user_id order by total_likes desc)
Morning Motivation	101	320	1
Workout Beats	101	250	2
Old Classics	101	150	3
Chill Vibes	102	180	1
Travel Tunes	102	175	2
Party Mix	103	450	1
Romantic Hits	104	210	1
LoFi Study	105	390	1

Task 5: Imagine you want to show the top 2 playlists per user based on total_likes, like Spotify's 'Your Top Playlists' feature. Write a query using a window function to select only the top 2 playlists for each user.

Query / Answer:

```
WITH RankedPlaylists AS (  
  SELECT  
    playlist_name,  
    user_id,  
    total_likes,  
    ROW_NUMBER() OVER (  
      PARTITION BY user_id  
      ORDER BY total_likes DESC  
    ) AS rn  
  FROM Playlists  
)  
SELECT playlist_name, user_id, total_likes  
FROM RankedPlaylists  
WHERE rn <= 2;
```

Hard need to ask ::--

Image / Screenshot: